

MONGODB LIBRARY MANAGEMENT SYSTEM

NoSQL, Data Modeling & CRUD Operations

Assignment Report

GitHub Repository

<https://github.com/khushipg04-tech/version-control-testing-project>

A practical study of MongoDB, NoSQL data modeling, installation, CRUD operations, queries, indexing, aggregation, and a library database example.

1. Introduction

MongoDB is a NoSQL, document-oriented database that stores data in flexible BSON documents. Unlike relational databases that mainly organize information into tables and rows, MongoDB uses collections and documents. This assignment demonstrates MongoDB through a simple Library Management System containing books, authors, and genres.

2. Objectives

Understand NoSQL and MongoDB concepts; learn installation and configuration; design a library database; perform Create, Read, Update, and Delete operations; write search queries for books and authors; and understand indexing, aggregation, and flexible schema design.

3. MongoDB and NoSQL Concepts

MongoDB is classified as a NoSQL database. Its documents are stored internally in BSON format and can contain nested objects and arrays. A database contains collections, and collections contain documents. A document is roughly equivalent to a row in a relational database, while a collection is roughly equivalent to a table.

4. Installation and Configuration

MongoDB can be installed using the appropriate official installer or package repository for the operating system. The MongoDB server process is mongod. The default server port is 27017. Modern MongoDB versions use mongosh as the command-line shell.

5. Library Database Design

The database is named libraryDB. It contains books, authors, and genres collections. A book document stores fields such as title, author, genre, publication year, and availability.

```
use libraryDB

db.books.insertMany([
  { title: "The Alchemist", author: "Paulo Coelho",
    genre: "Fiction", year: 1988, available: true },
  { title: "Clean Code", author: "Robert C. Martin",
    genre: "Technology", year: 2008, available: true },
  { title: "Atomic Habits", author: "James Clear",
    genre: "Self-Help", year: 2018, available: false }
])

db.authors.insertMany([
  { name: "Paulo Coelho", country: "Brazil" },
  { name: "Robert C. Martin", country: "USA" },
  { name: "James Clear", country: "USA" }
])

db.genres.insertMany([
  { name: "Fiction" },
  { name: "Technology" },
  { name: "Self-Help" }
])
```

6. CRUD Operations

CRUD stands for Create, Read, Update, and Delete. These are the four basic operations for managing database data.

6.1 Create

```
db.books.insertOne({
```

```

    title: "1984",
    author: "George Orwell",
    genre: "Fiction",
    year: 1949,
    available: true
  })

```

6.2 Read

```

db.books.find()
db.books.findOne({ title: "Clean Code" })

```

6.3 Update

```

db.books.updateOne(
  { title: "Clean Code" },
  { $set: { available: false } }
)

```

6.4 Delete

```

db.books.deleteOne({ title: "1984" })
db.books.deleteMany({ available: false })

```

7. Searching Books

MongoDB queries can search books using author, genre, publication year, availability, or combinations of conditions.

```

// By author
db.books.find({ author: "James Clear" })

// By genre
db.books.find({ genre: "Technology" })

// Published after 2000
db.books.find({ year: { $gt: 2000 } })

// Available books
db.books.find({ available: true })

// Multiple conditions
db.books.find({ genre: "Fiction", available: true })

```

8. Searching Authors

```

// Find one author
db.authors.findOne({ name: "Paulo Coelho" })

// Authors from USA
db.authors.find({ country: "USA" })

// Names starting with James
db.authors.find({ name: /^James/ })

```

9. Indexing

Indexes improve query performance by allowing MongoDB to locate matching documents efficiently. Indexes should be created according to actual query patterns rather than indiscriminately indexing every field.

```

db.books.createIndex({ title: 1 })
db.books.createIndex({ author: 1 })

db.books.find({ title: "Clean Code" })
  .explain("executionStats")

```

10. Aggregation Example

The aggregation framework can process documents and produce summaries. The following example counts books by genre and sorts the result.

```
db.books.aggregate([
  { $group: {
    _id: "$genre",
    totalBooks: { $sum: 1 }
  }},
  { $sort: { totalBooks: -1 } }
])
```

11. Data Modeling Approach

MongoDB schema design should be based on application query patterns. Related information can be embedded when it is commonly accessed together, while references can be used when related data is large, shared, or independently managed. In this simple library example, author and genre names are stored directly in book documents to make common searches straightforward.

12. Importance of NoSQL in Modern Applications

NoSQL databases are useful for applications that need flexible data structures, large-scale storage, high traffic, or distributed workloads. MongoDB can accommodate changing data models and supports features such as indexing, aggregation, replication, and sharding. It is commonly suitable for catalogs, content systems, analytics, and other applications where flexible documents are useful.

13. Findings and Reflection

The Library Management System shows that MongoDB can represent application data naturally as documents. CRUD operations are straightforward, and queries can filter documents using fields and comparison operators. Indexing can improve frequently used searches, while aggregation can transform stored information into useful summaries.

The main learning is that NoSQL does not mean that data has no structure. Instead, MongoDB allows developers to design a flexible structure that matches application requirements and query patterns.

14. Testing Checklist

- Create the libraryDB database and collections.
- Insert books, authors, and genres.
- Retrieve all and individual books.
- Search books by author, genre, year, and availability.
- Update a book and verify the change.
- Delete a document and verify its removal.
- Create indexes and inspect query execution.
- Run aggregation to summarize books by genre.

15. Learning Outcomes

- Explain MongoDB and NoSQL concepts.
- Understand documents and collections.
- Perform MongoDB CRUD operations.
- Write search queries using filters and comparison operators.
- Use indexes to improve query performance.

- Use aggregation for basic analysis.
- Design a simple database around application query requirements.

16. Conclusion

MongoDB provides a flexible document-based approach to database management. The Library Management System demonstrates installation concepts, data modeling, CRUD operations, search queries, indexing, and aggregation. NoSQL databases such as MongoDB are important in modern applications because they support flexible data models and scalable architectures.

17. Submission Details

Title: MongoDB Library Management System – NoSQL, Data Modeling & CRUD Operations

Link: <https://github.com/khushipg04-tech/version-control-testing-project>

PDF: MongoDB_Library_Management_Assignment.pdf